课 程 内 容

第 1 章 编 译 引 论

第 2 章 形式语言与自动机基础

第 3 章 词法分析 (Lexical Analysis)

第 4 章 语法分析(Syntax Analysis)
—— 自上而下分析法

第 5 章 语法分析(Syntax Analysis)
—— 自下而上分析法

第 6 章 语义分析与中间代码生成

第 7 章 运 行 环 境

第 8 章 代码优化（optimization）

第7章 运行环境

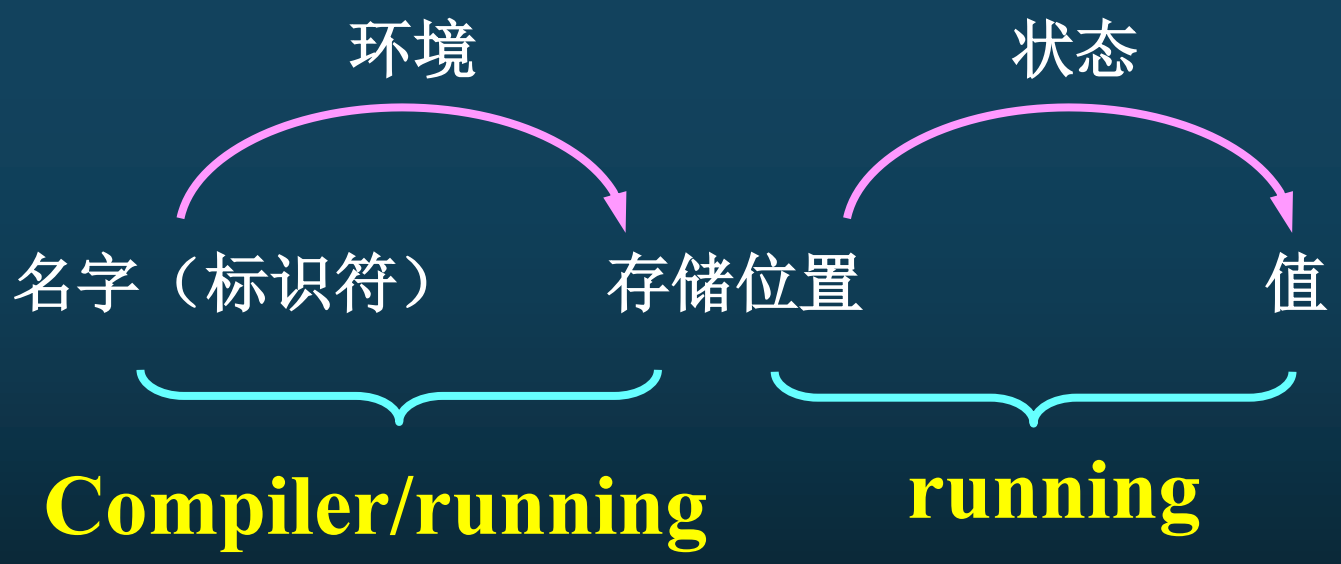
7.1 关于程序运行环境与存储组织

7.2 静态运行时环境与存储分配

7.3 基于栈的运行时环境的动态存储分配

- 运行环境

指目标计算机的寄存器及存储器结构，用来管理存储器并保存执行过程所需的信息。



✌ 注意：

* 环境、状态的区别：

赋值改变状态，但不改变环境。

运行环境
- 完全静态环境 (full static environment)
- 基于栈的环境 (stack_based environment)
- 完全动态环境 (full dynamic environment)

* 环境与语言特性相关：

FORTRAN完全静态环境；

C，C++，PASCAL，Ada基于栈的环境；

LISP完全动态环境。

- **静态：** 如果一个名字的性质通过说明语句的隐或显规则而定义，则称这种性质是“静态”确定的。

- **动态：** 如果名字的性质只有在程序运行时才能知道，则称这种性质为“动态”确定的。

存储分配策略
- 静态分配策略
- 动态分配策略
  - 栈式方案
  - 堆式方案

- 静态存储分配

在编译时能够确定目标程序运行时所需的全部数据空间的大小，即在编译时就可以将程序中的名字关联到存储单元，确定其存储位置，这种分配策略称为静态存储分配。

- 动态存储分配

在编译时不能确定目标程序运行时所需的全部数据空间的大小，而是在目标程序运行时动态确定的。

- 栈式动态存储分配

在内存中开辟一个栈区，按栈的特性进行存储分配。程序运行时，每当进入一个函数或过程，该函数所需的存储空间动态地分配于栈顶，函数返回时，释放所占用的空间。

堆式动态存储分配

在内存中开辟一个称为堆的存储区，程序运行每当需要（申请）时就按照某种分配原则在堆的自由区(可占用区)中，分配能满足其需要的存储空间给它，使用后需要释放操作，再将不再占用的存储空间归还给堆的自由区。

🔔 运行时存储分配的一个原则是，尽可能对数据对象进行静态分配 。

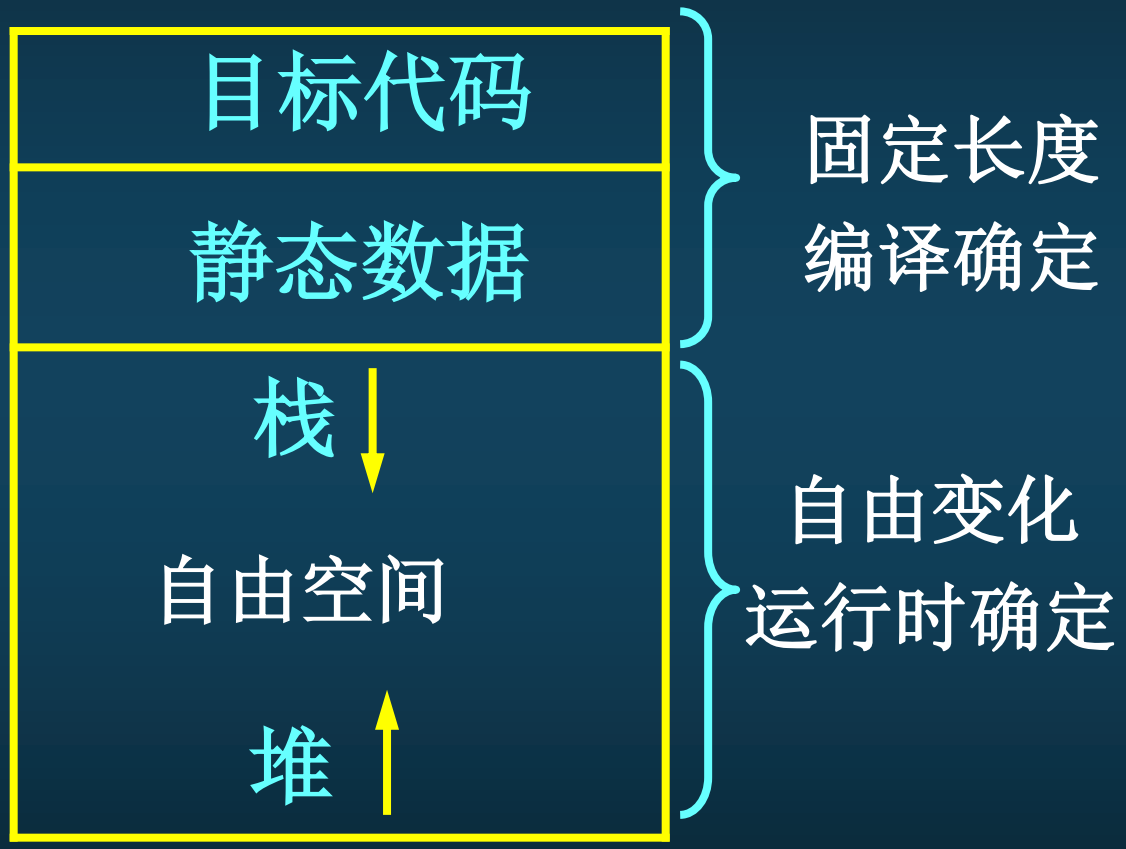
存储区保存的对象

(1) 生成的目标代码。(代码区)

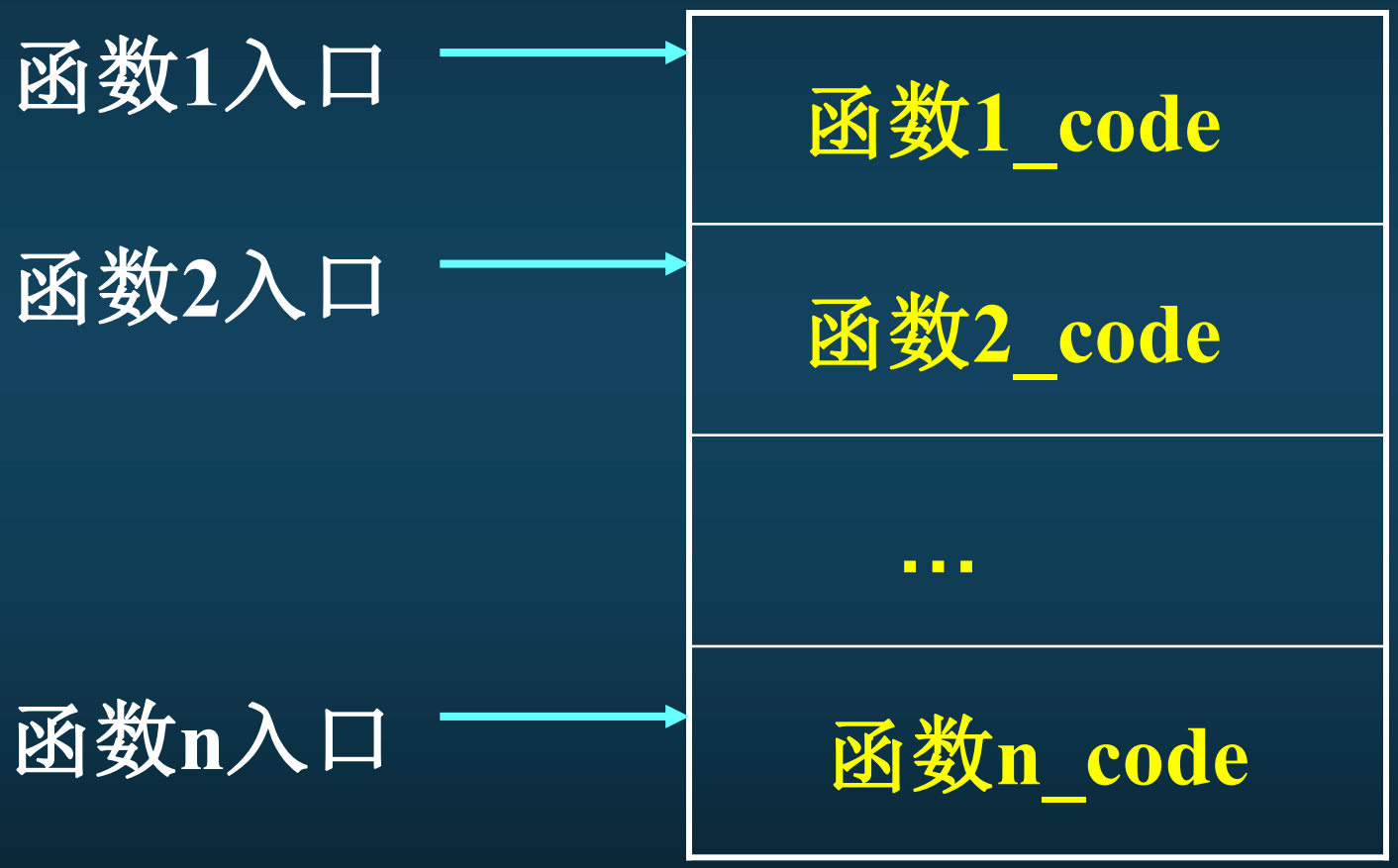
(2) 目标代码运行时的数据空间。包括用户定义的各种类型的数据对象，作为保存中间结果和传递参数的临时工作单元，组织输入、输出所需的缓冲区等。

(3) 记录过程活动的控制栈。

- 运行时内存分成代码区和数据区的典型划分



- 代码区



- 存储分配的重要单元—过程的活动记录AR（Procedure Activation Record）

一块连续的存储区，被用来存放过程或函数的一次调用执行所需要的信息。

活动记录的组织依赖于目标机体系结构，被编译的语言特性等。

函数调用的语义处理：

① 检查所调用的过程或函数是否定义；与所定义的过程或函数的类型、实参与形参的数量及类型是否一致；

② 给被调过程或函数分配活动记录所需的存储空间；

③ 计算并传送实参；

④ 加载调用结果和返回地址，恢复主调用过程或函数的继续执行；

⑤ 转向相应的过程或函数。

- 活动记录基本内容

自变量实参空间
局部数据空间
局部临时变量空间
机器状态信息
返回地址 RA
存取链 SL
控制链 DL

第 7 章 运 行 环 境

7.1 关于程序运行环境与存储组织

7.2 静态运行时环境与存储分配

7.3 基于栈的运行时环境的动态存储分配

静态存储分配

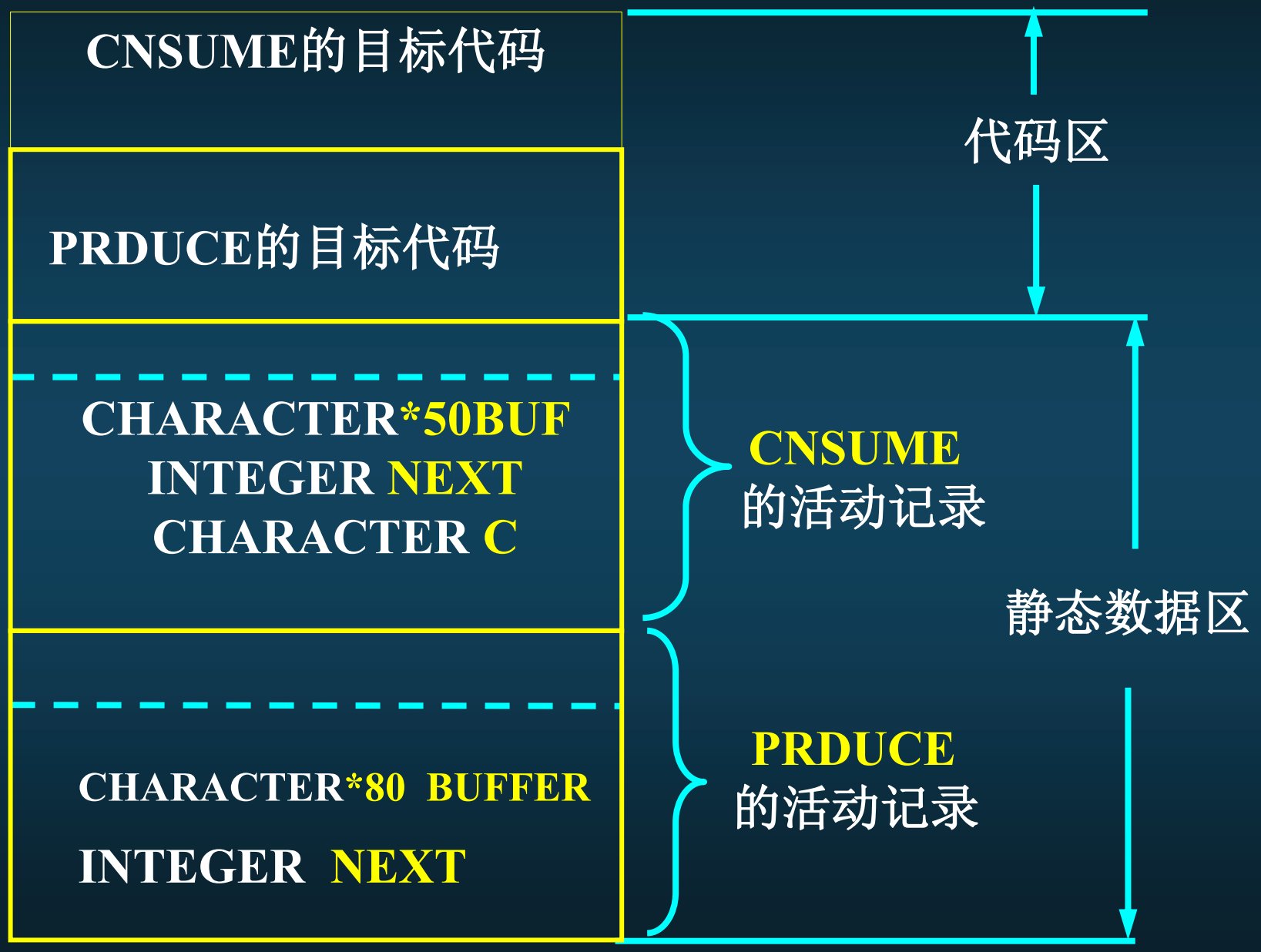
在编译时能够确定目标程序运行时所需的全部数据空间的大小，即在编译时就可以将程序中的名字关联到存储单元，确定其存储位置，这种分配策略称为静态存储分配。

例7.1 给出FORTRAN程序如下。

```
(1)        PROGRAM CNSUME
(2)              CHARACTER *50 BUF
(3)              INTEGER   NEXT
(4)              CHARACTER C, PRDUCE
(5)              DATA NEXT / 1 / ,BUF/ ' ' /
(6)        6     C=PRDUCE( )
(7)              BUF(NEXT:NEXT)=C
(8)              NEXT=NEXT+1
(9)              IF(C.NE.' ')  GOTO 6
(10)             WRITE(* , '(A)' ) BUF
(11)          END
```

例7.1 给出FORTRAN程序如下。

```
(12)    CHARACTER FUNCTION PRDUCE()
(13)        CHARACTER   *80  BUFFER
(14)        INTEGER NEXT
(15)        SAVE BUFFER , NEXT
(16)        DATA NEXT /81/
(17)        IF (NEXT.GT.80) THEN
(18)        READ ( * , '(A)' ) BUFFER
(19)        NEXT=1
(20)        END IF
(21)        PRDUCE=BUFFER(NEXT:NEXT)
(22)        NEXT=NEXT+1
(23)    END
```


CNSUME的目标代码
PRDUCE的目标代码
CHARACTER*50BUF
INTEGER NEXT
CHARACTER C
CHARACTER*80 BUFFER
INTEGER NEXT
代码区
CNSUME
的活动记录
静态数据区
PRDUCE
的活动记录

静态分配对语言的限制

(1) 数据对象的长度和它在内存中的位置的限制必须在编译时知道。

(2) 不允许递归过程，因为一个过程的所有活动使用同样的局部名字结合。

(3) 数据结构不能动态建立，因为没有运行时的存储分配机制。

第 7 章 运 行 环 境

7.1 关于程序运行环境与存储组织

7.2 静态运行时环境与存储分配

7.3 基于栈的运行时环境的动态存储分配

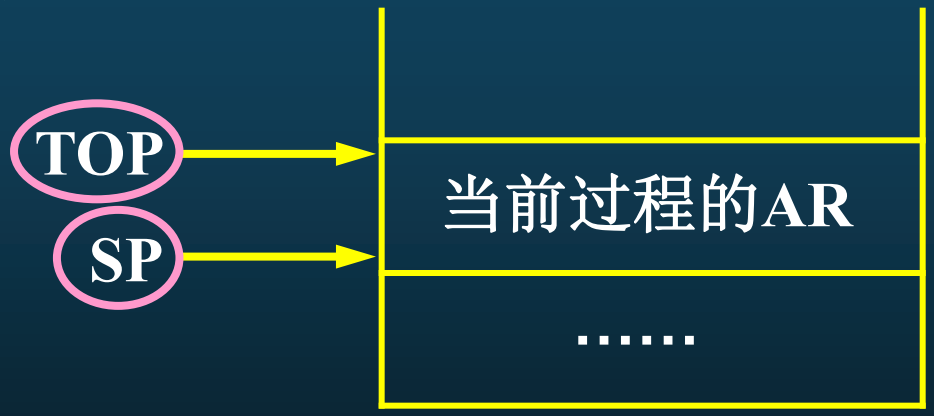
基于栈的运行时环境的动态存储分配

将整个程序运行时使用的存储空间都安排在一个栈里。每当调用一个函数时，它所需的数据空间就分配在栈顶，每当过程结束时就释放这部分空间。

函数所需的数据空间
- 生存期在本过程的本次活动中的数据对象 (局部变量、临时变量…)
- 管理过程活动的记录信息 (过程调用中断时当前机器的状态信息)

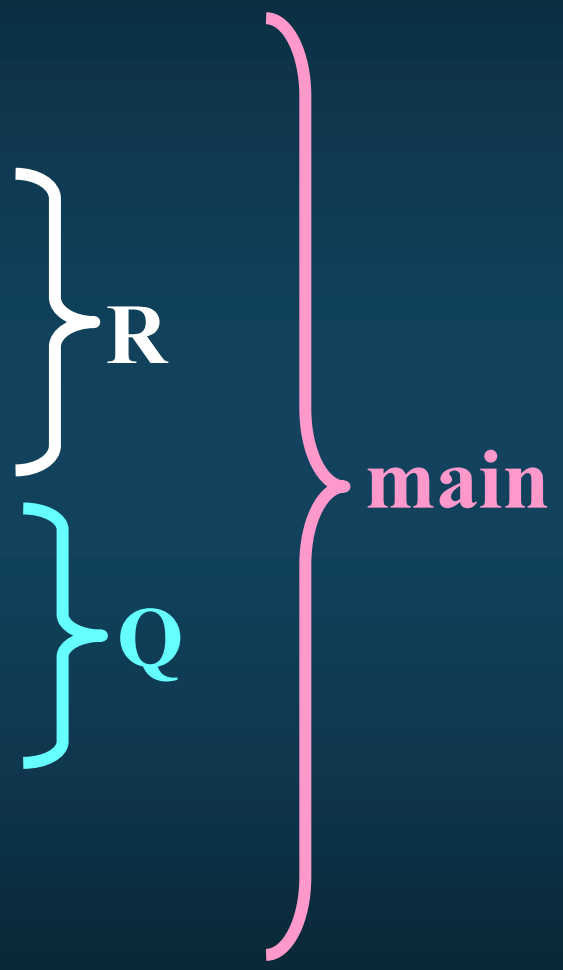
活动记录的栈

运行时存储活动记录的空间。称为运行时栈或调用栈。它随着程序执行时发生的调用链生长或缩小。每个过程每次在调用栈上可以有不同的活动记录，每个都代表一个不同的调用。

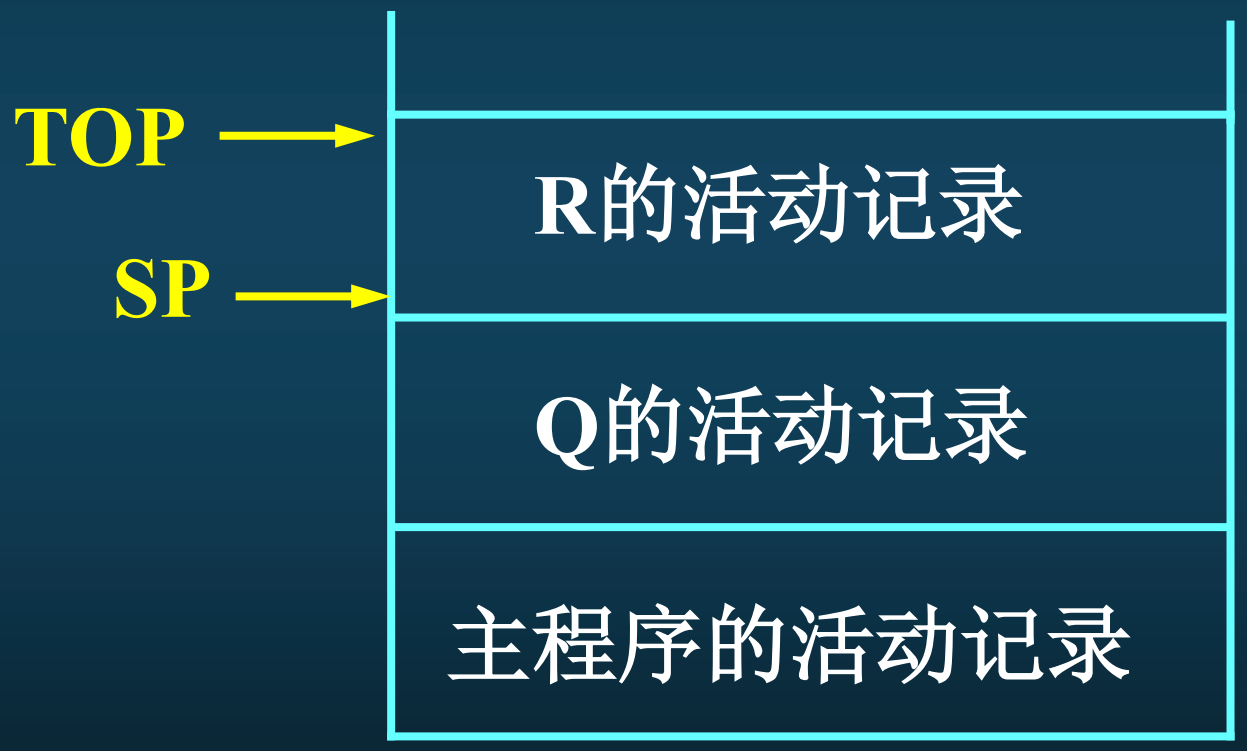


例7.2 设有如下程序(允许嵌套定义过程）

```
main
    全局变量的说明
        proc   R
          ......
        end  R;
        proc  Q
          ......
        end  Q;
    主程序执行语句
end  main
```

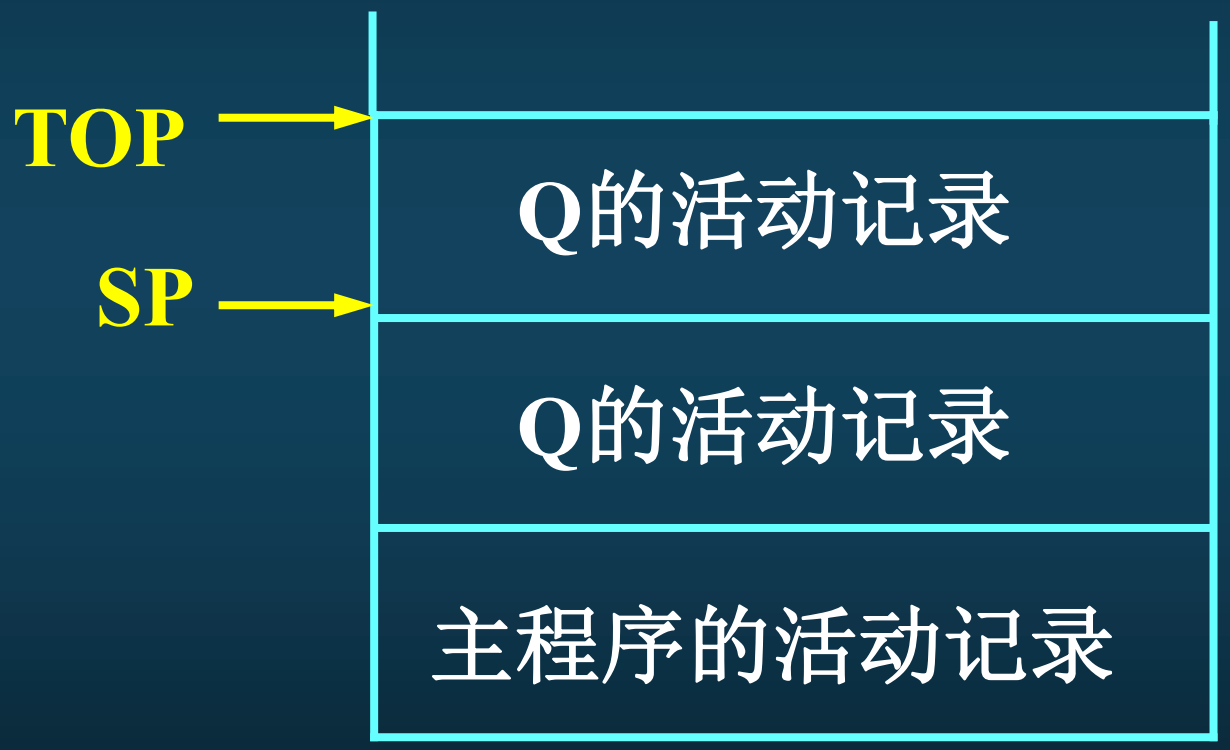


程序运行顺序： Main $\xrightarrow{\text{调用}}$ Q $\xrightarrow{\text{调用}}$ R



活动记录的栈

程序运行顺序： Main $\xrightarrow{\text{调用}}$ Q $\xrightarrow{\text{调用}}$ Q



活动记录的栈

嵌套定义的过程可使用的数据：

1、本过程定义的数据；

2、外层过程中定义的数据。

引入问题：

外层过程中定义的数据怎样找到？

- 关键技术：

  解决对非局部量的引用（存取）。

  设法跟踪每个外层过程的最新活动记录AR的位置。

- 跟踪办法：

  1. 用静态(访问、存取）链。

  2. 用DISPLAY表。

AR（活动记录）

局部变量
机器状态信息
存 取 链 → SL
控 制 链 → DL
实　参
返回地址

SL：存取链。指向定义该过程的直接外层过程运行时最新活动记录的基地址。

DL：控制链。 指向调用该过程前正在运行过程的活动记录基地址。

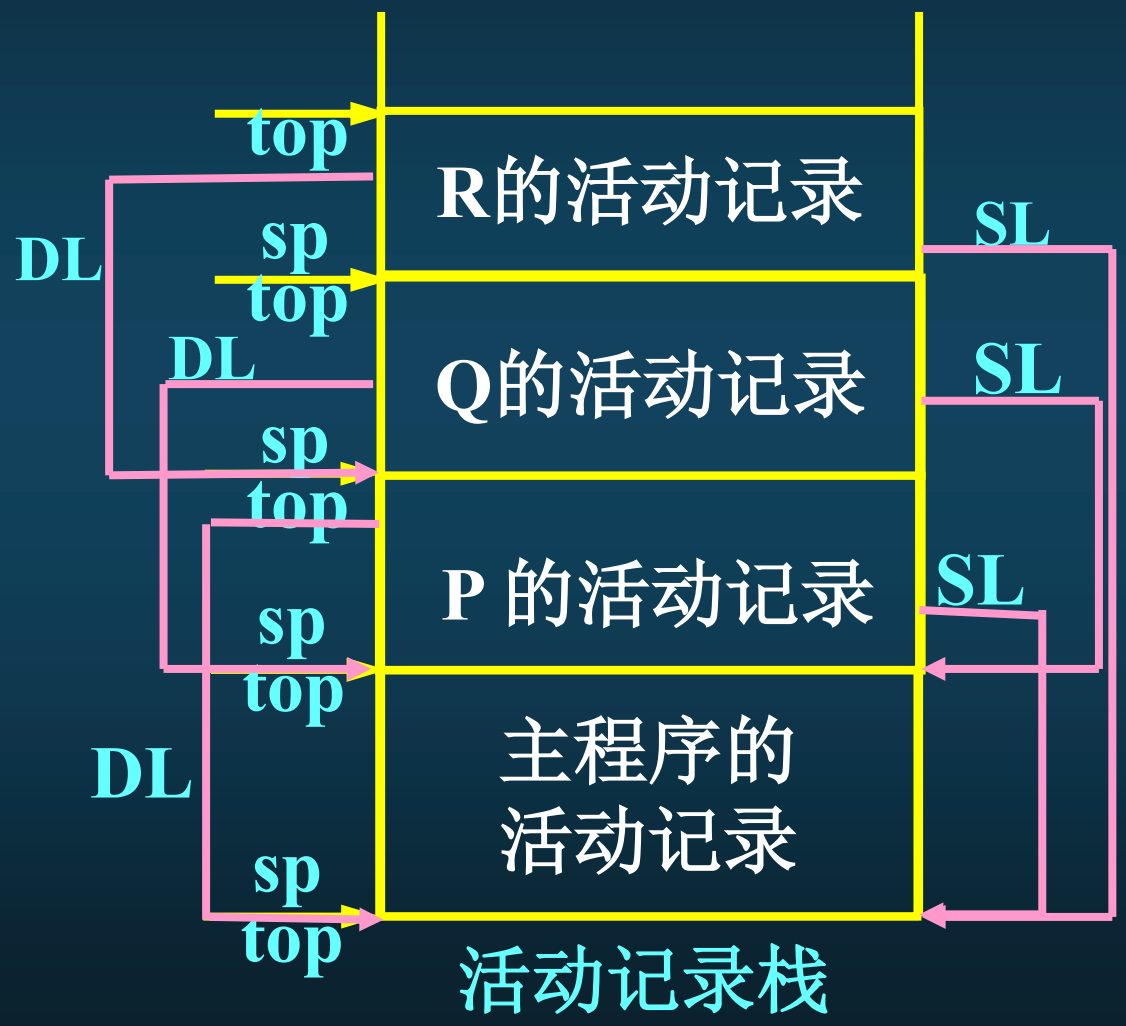
例7.3

```
program  main( i,0) ;
        ……
    proc  R(c,d) ;
        ……
    end  /*R*/
   proc    P (a);
     ……
       proc   Q (b);
            ……
          R(x,y);
       end    /* Q*/
     ……
      Q(z);
    end   /* P*/
     ……
   P(W);
     ……
   R(U,V);
     ……
end   /* main*/
```

程序结构图

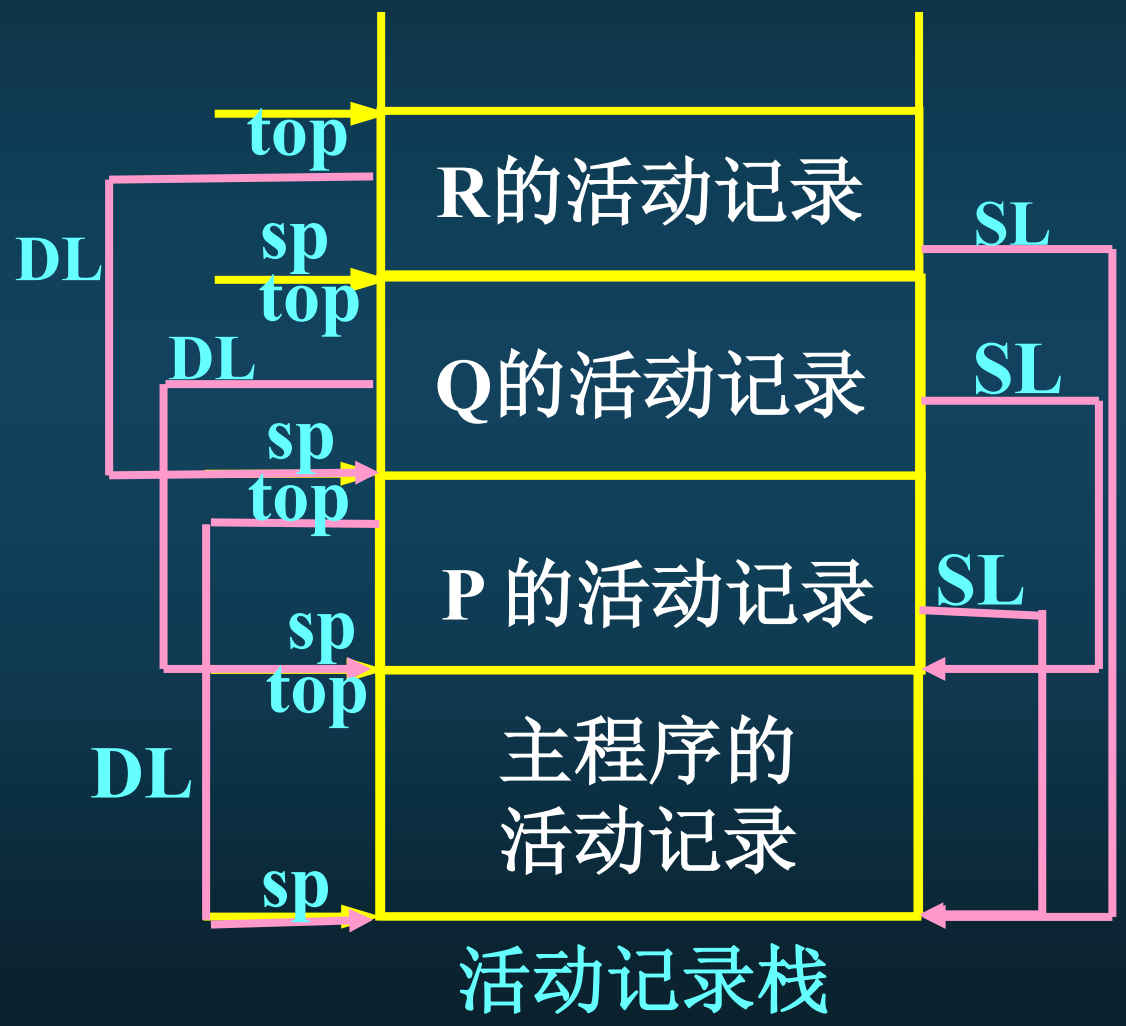
用SL(存取链）的方案

主程序 ➔ P ➔ Q ➔ R



用SL(存取链）的方案

主程序 ➔ P ➔ Q ➔ R



用SL(存取链）的方案

主程序 ➔ R

例7.4 设有C程序

gcd(15,10)

↓

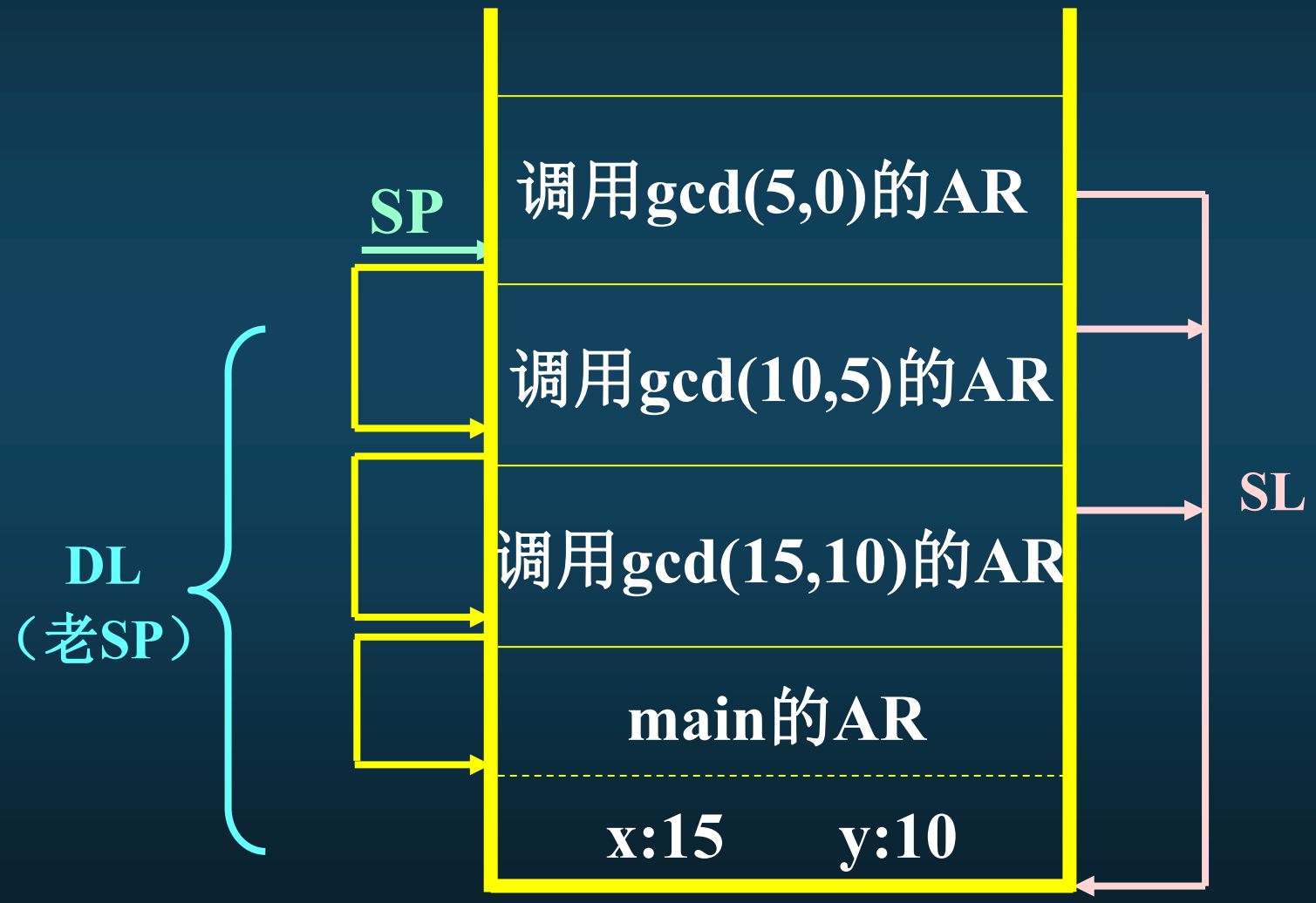
gcd(10,5)

↓

gcd(5,0)

```
#include <stdio.h>
int x,y ;
int gcd(int u,int v)
{  if (v==0) return u;
    else return gcd(v, u%v);
}
main()
{ scanf("%d%d",&x,&y);
   printf("%d\n",gcd(x,y));
   return;
}
```

C语言中无过程（函数）的嵌套定义，所以数据链简单，同指向全局数据。



- 解决对非局部量的引用：用Display表

Display表 — 嵌套层次显示表

主程序设为第0层，主程序中定义的过程为第1层，第i层过程中定义的过程为i＋1层。假设当前激活过程的层次为K，它的Display表含有K+1个单元，依次存放着现行层，直接外层…直至最外层的每一过程的最新活动记录的基地址。

构造display区的规则：

假定从i层过程进入第j层过程，则有：

① 如果j=i+1 (即调用当前块局部说明的过程块)，则复制第i层的display;转③。

② 如果 j≤i (即调用对当前过程来说属于全程说明的过程)，则复制来自i层过程活动记录中的display区前面j个记录；转③。

③ 再增加一个指向j层过程活动记录基的指针；

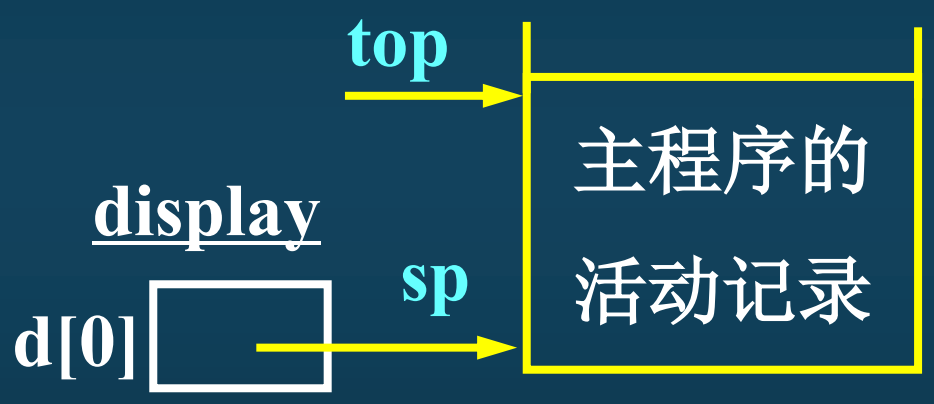
例7.3

```
program  main( i,0) ;
        ……
    proc  R(c,d) ;
        ……
    end  /*R*/
   proc    P (a);
     ……
       proc   Q (b);
            ……
          R(x,y);
       end    /* Q*/
     ……
      Q(z);
    end   /* P*/
     ……
   P(W);
     ……
   R(U,V);
     ……
end   /* main*/
```

程序结构图

用Display表的方案

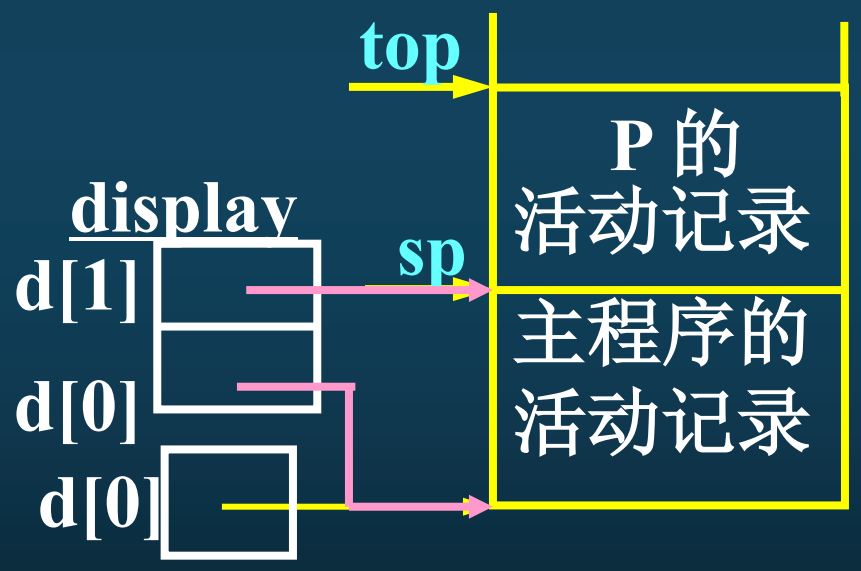
主程序 (1) ➔ P(2) ➔ Q(3) ➔ R(4) ➔ …



（1）

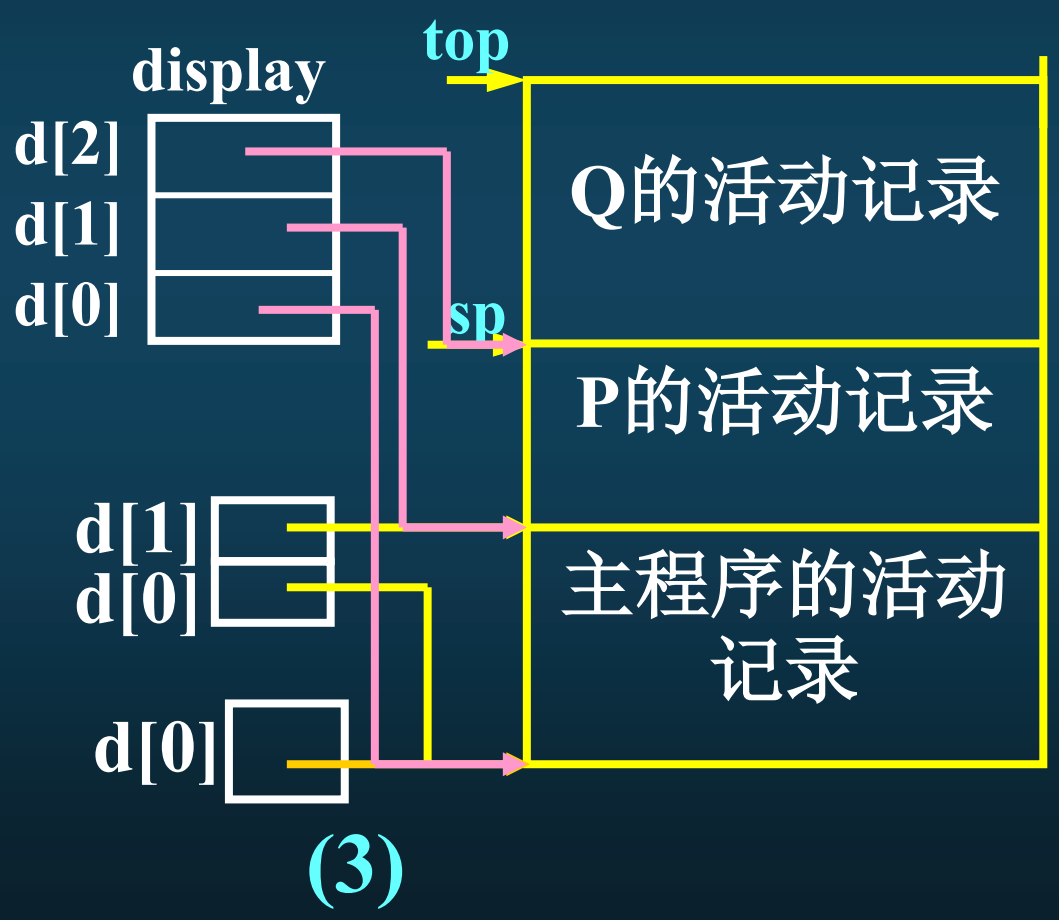
- 用Display表的方案

主程序 (1) ➔ P(2) ➔ Q(3) ➔ R(4) ➔ …



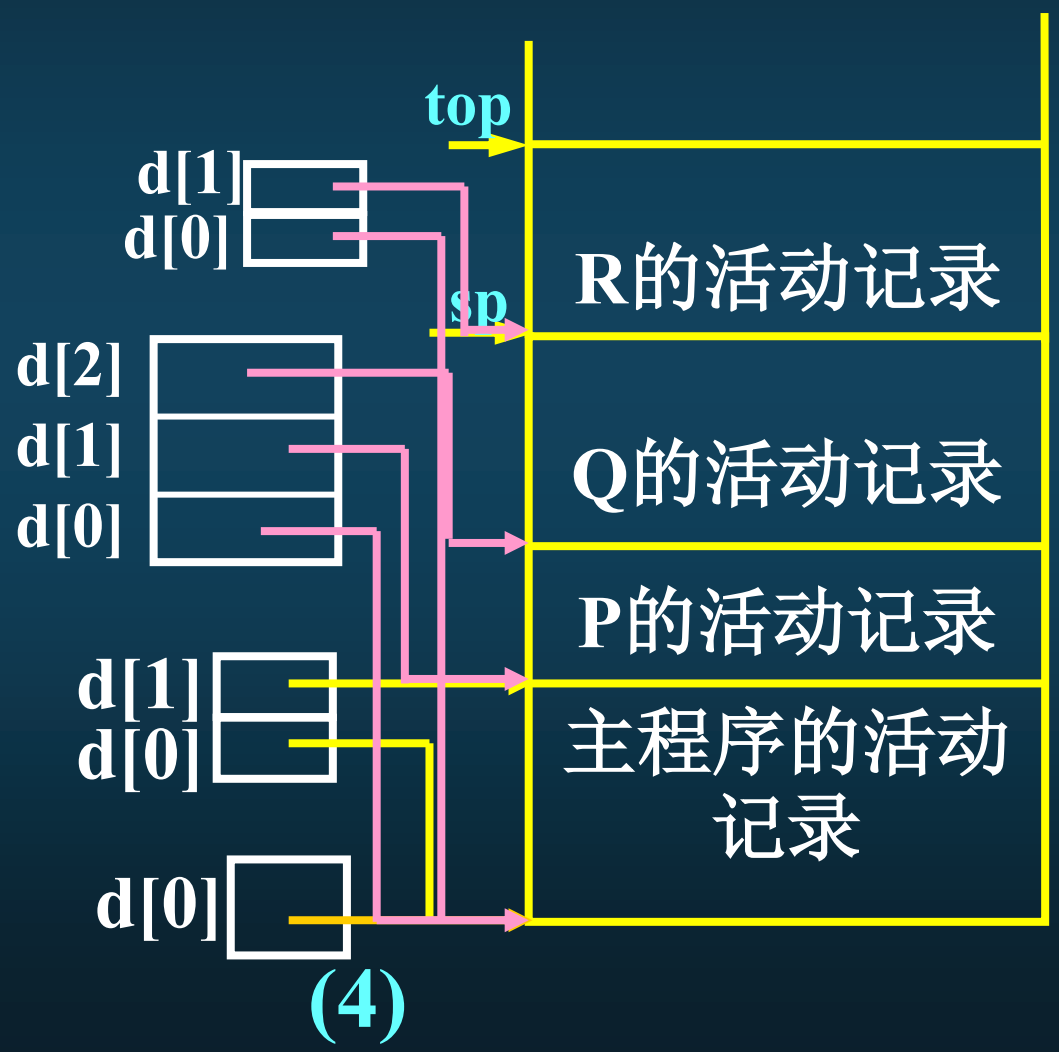
（2）

主程序 (1) ➔ P(2) ➔ Q(3) ➔ R(4)


display
d[2]
d[1]
d[0]
top
sp
Q的活动记录
P的活动记录
主程序的活动记录
d[1]
d[0]
d[0]
(3)

主程序
R
P
Q
call R
call Q
call P
call R

主程序 (1) ➔ P(2) ➔ Q(3) ➔ R(4)


top
sp
d[1]
d[0]
d[2]
d[1]
d[0]
d[1]
d[0]
d[0]
(4)
R的活动记录
Q的活动记录
P的活动记录
主程序的活动记录

主程序
R
P
Q
call R
call Q
call P
call R

P219:7-7为以下的Pascal程序画出活动记录的栈。

(1)对p的第一次调用中对a的调用之后。

(2)在对p的第二次调用中对a的调用之后。

(3)程序打印出什么？为什么？

程序结构图

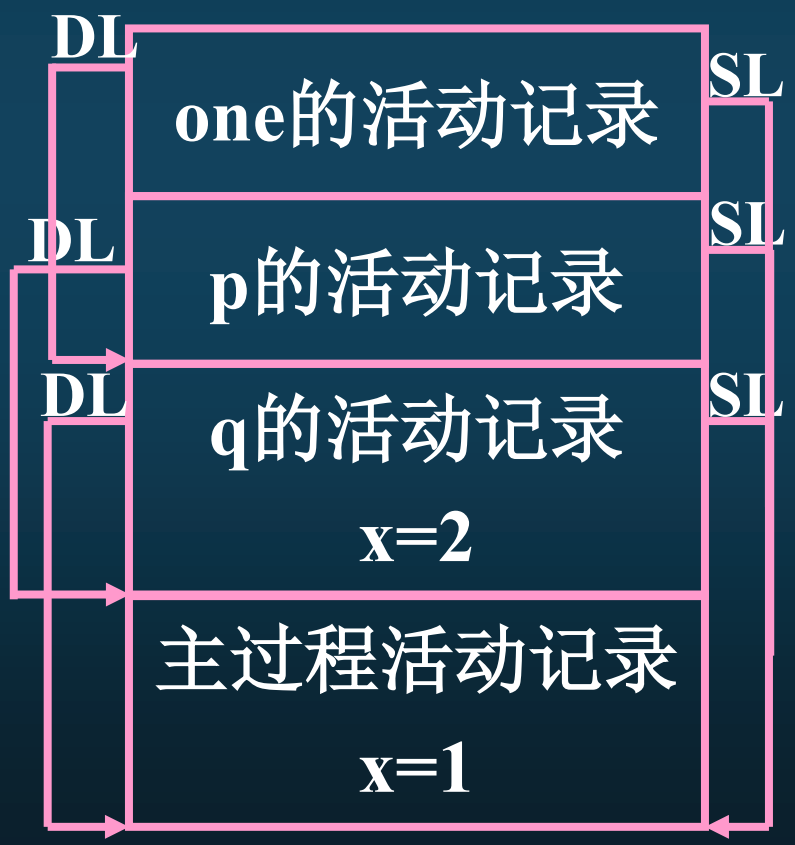
```
PROGRAM closureEx (Output);
  VAR x:Integer;
  PROCEDURE one;
  BEGIN
    Writeln(x);
  END;
  PROCEDURE p(PROCEDURE a);
  BEGIN
   a;
  END;
  PROCEDURE q;
  VAR x:Integer;
     PROCEDURE two;
     BEGIN
       Writeln(x);
     END;
  BEGIN
   x : = 2;
   p (one);
   p (two);
  END;  (* q *)
  BEGIN  (* main *)
   x : = 1;
   q;
  END.
```

P219:7-7为以下的Pascal程序画出活动记录的栈。

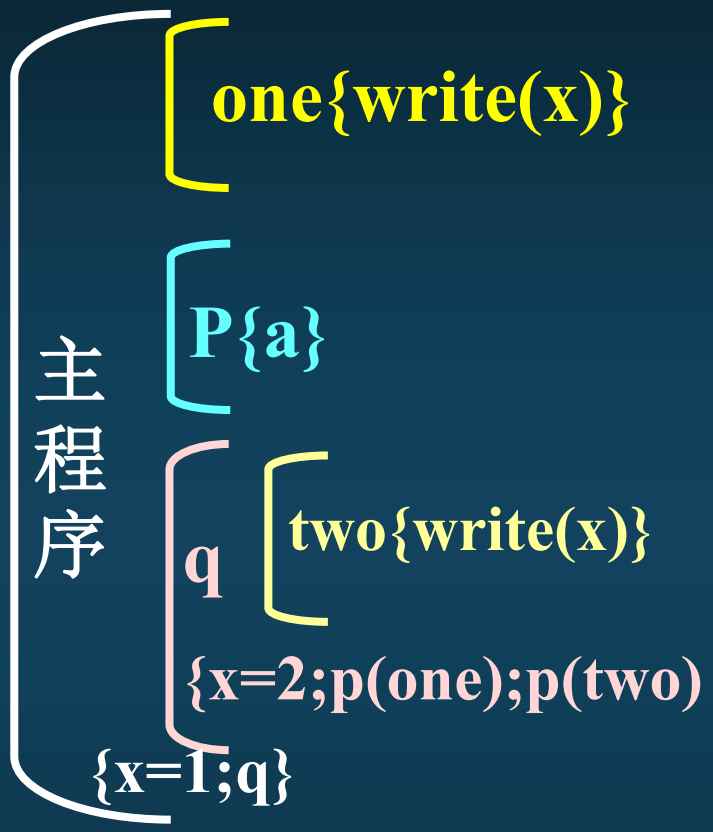
(1)对p的第一次调用中对a的调用之后。

(2)在对p的第二次调用中对a的调用之后。

(3)程序打印出什么？为什么？



程序结构图



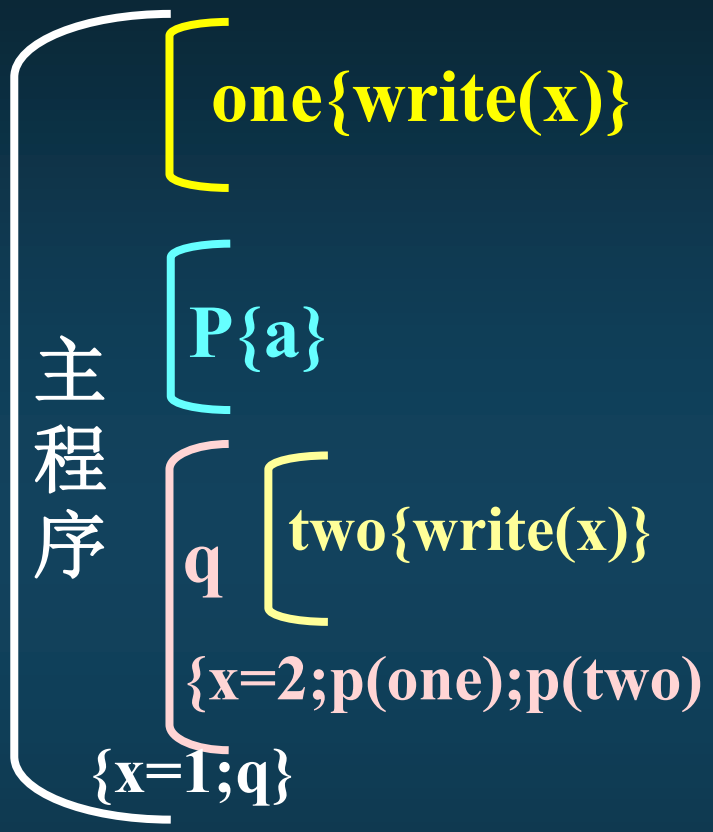
P219:7-7为以下的Pascal程序画出活动记录的栈。

(1)对p的第一次调用中对a的调用之后。

(2)在对p的第二次调用中对a的调用之后。

(3)程序打印出什么？为什么？

程序结构图



运行环境　　总结

三种运行环境及三种分配方案

概念：活动记录

以过程为单位的栈式动态分配方案

解决非局部变量的引用查找的两种方式：

数据链SL；

嵌套层次显示表Display。

第七章 结束